

FOSS Security Tools:

Binwalk

In this fifth article in the FOSS security series, we will explore the use of the binwalk tool.

Binwalk is used to scan firmware for embedded files and executable code. This includes compressed files, firmware headers, archive files, bootloaders, file systems, etc. It is supported on GNU/Linux, Mac OS X, Windows, and FreeBSD. An API allows you to access its classes and methods to write your own binwalk scripts. The source code is written in Python and released under the MIT licence.

Installation

A Parabola GNU/Linux-libre (x86_64) system is used to install binwalk and its dependency packages.

```
$ cat /etc/os-release
NAME=Parabola
PRETTY_NAME="Parabola GNU/Linux-libre"
ID=parabola
ID_LIKE=arch
BUILD_ID=rolling
VARIANT="x86_64 SystemD Edition"
VARIANT_ID="x86_64-systemd"
ANSI_COLOR="1;35"
HOME_URL="https://www.parabola.nu/"
DOCUMENTATION_URL="https://wiki.parabola.nu/"
SUPPORT_URL="irc://irc.libera.chat/#parabola"
BUG_REPORT_URL="https://labs.parabola.nu/"
LOGO="parabola-logo"
```

You can use the *pacman* package manager to install binwalk as shown below:

```
$ sudo pacman -S binwalk squashfs-tools python-matplotlib
python-gobject
```

The *squashfs-tools* package is needed during the extraction of a *squashfs* file system present in firmware. The *python-matplotlib* and *python-gobject* packages are required for producing entropy charts.

Help

Binwalk version 2.4.1 is installed at the time of writing this article

on Parabola GNU/Linux-libre. You can view the command line options using the ‘-h’ help option as shown below:

```
$ binwalk -h
```

```
Binwalk v2.4.1
```

```
Original author: Craig Heffner, ReFirmLabs
```

```
https://github.com/OSPG/binwalk
```

```
Usage: binwalk [OPTIONS] [FILE1] [FILE2] [FILE3] ...
```

```
Signature Scan Options:
```

```
-B, --signature Scan target file(s) for common file signatures
-R, --raw=<str> Scan target file(s) for the specified sequence
                    of bytes
-A, --opcodes Scan target file(s) for common executable
                    opcode signatures
```

```
...
```

Signature

Consider the Buffalo WHR-G125 2007 router firmware available at the *dd-wrt.com* website. You can view the firmware signature with the ‘-B’ option as follows:

```
$ binwalk -B dd-wrt.v24_whr-g125.bin
```

DECIMAL	HEXADECIMAL	DESCRIPTION
0	0x0	TRX firmware header, little endian, image size: 3543040 bytes, CRC32: 0x85472C8C, flags: 0x0, version: 1, header size: 28 bytes, loader offset: 0x1C, linux kernel offset: 0x8E4, rootfs offset: 0x8E7CC
28	0x1C	gzip compressed data, maximum compression, from Unix, last modified: 1970-01-01 00:00:00 (null date)
2276	0x8E4	LZMA compressed data, properties: 0x5D, dictionary size: 8388608 bytes, uncompressed size: 1982464 bytes
583628	0x8E7CC	Squashfs filesystem, little endian, DD-WRT signature, version 3.0, size: 2954340 bytes,